ARQUITETURA DE SOFTWARE NA PLATAFORMA NODE.JS

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

O QUE É O NODE.JS 5

O SURGIMENTO DO NODE.JS 7

CARACTERÍSTICAS DO NODE.JS 8

VANTAGENS DO USO DO NODE.JS 12

PRINCÍPIOS 14

ESTRUTURA E PERFORMANCE 16

ESCALABILIDADE 17

ESTENSIBILIDADE E PACOTES 18

ARQUITETURA LIMPA 19

EXPRESS 21

A IMENSIDÃO DO NODE.JS 22

CASOS DE USO MAIS COMUNS 23

CONSIDERAÇÕES FINAIS 25

REFERÊNCIAS 26

INTRODUÇÃO

É natural que o ambiente mais propício para o desenvolvimento de sistemas nesse novo paradigma, por sua natureza distribuída, seja a Internet. Dado que a linguagem mais utilizada hoje na Web é o JavaScript, implementado na maioria dos navegadores atuais, se deu a escolha dessa linguagem para o projeto.

Estando o JavaScript em franca expansão também no server side, com a implementação do Node.js , usado hoje por grandes organizações em suas soluções Web, essa também foi a linguagem utilizada no lado servidor do projeto.

Com o uso da mesma linguagem em todo o projeto, o mesmo ganhou coesão e houve redução na tradicional lacuna que separa os desenvolvimentos front end e back end. A importância da integração de sistemas em rede trouxe à tona a necessidade de uma arquitetura que defina como esses sistemas devem interagir entre si e como devem definir suas interfaces. Um dos estilos arquiteturais que se propõe a resolver essa questão é o REST (Transferência de Estado Representacional).

O QUE É O NODE.JS

O Node.JS é uma plataforma leve para você construir aplicações servidoras em linguagem Javascript e pode ser comparado, em um primeiro momento de análise, a tecnologias como o Java EE Web ou ASP.NET. Ao mesmo tempo, Node.JS é uma tecnologia minimalista. Ele é muito leve e não traz consigo servidores Web ou servidores de aplicação pesados, como o Java EE Web ou o ASP.NET. No Node.JS, você começa leve e coloca novos módulos apenas quando necessário.

O JavaScript no lado do servidor pode ser um conceito novo para todos que trabalharam exclusivamente com o JavaScript no lado do cliente, mas a ideia em sí não é tão absurda porque não usar a mesma linguagem de programação no cliente que você usa no servidor?

Em nível detalhado, o Node.js é uma plataforma construída sobre o motor JavaScript do Google Chrome para construir aplicações de Internet rápidas e com excelente escalabilidade. O motor JavaScript V8 é o motor que a Google usa com seu navegador Chrome. A engine JavaScript do Chrome realmente interpreta o código JavaScript e o executa. Com o V8 a Google criou um interpretador ultrarrápido escrito em C++.

O Node.js usa um modelo de I/O direcionada a eventos assíncronos que o torna leve e eficiente, ideal para aplicações em tempo quase real com troca intensa de dados através de dispositivos distribuídos. Em termos arquiteturais, o Node é um servidor de programas e roda sobre um pequeno motor JavaScript.

Mas javascript não é só no Browser? Não mais. Node é feito sobre e engine da Google chamada V8 que roda no Chrome, o V8 interpreta o código javascript e dessa forma é processado no lado do servidor.

Não podemos dizer que node é um Framework nem um Middleware nem uma Linguagem, na verdade trata-se de um runtime Javascript. Com o crescimento exponencial de bibliotecas, recursos e da comunidade, podemos dizer que node tornou-se uma plataforma.

As aplicações Node.js são projetadas visando maximizar o troughput e a eficiência, usando operações de entrada e saída não bloqueantes, e eventos assíncronos. As aplicações Node.js rodam, cada uma, em apenas uma thread. O Node.js usa, internamente, a engine Google V8 JavaScript para executar o código e uma grande parte de seus módulos é escrito em JavaScript. Node.js contém, nativamente, uma biblioteca HTTP de lado servidor, tornando possível rodar um servidor web sem outra ferramenta, como por exempo o Apache ou Nginx.

Devido às características citadas acima, o Node.js se adapta perfeitamente para o uso em aplicações de tempo real como uso intensivo de dados e que rodem em dispositivos distribuídos. Com um vasto ecossistema de bibliotecas, larga comunidade open-source, alta performance e facilidade de deploy, Node.js é uma excelente alternativa para a criação de servidores para aplicativos Web, trazendo, ainda, a vantagem de se poder utilizar a mesma linguagem no server side e no front end.

O SURGIMENTO DO NODE.JS

Apesar do Javascript ter mais de 20 anos, o seu uso server-side é bem recente.

A linguagem Javascript foi criada em 1995, e se tornou a linguagem padrão dos browsers e consequentemente da *Web* para o desenvolvimento client-side.

Desde então, houveram diversas tentativas de implementar sua execução server-side. Todas elas fracassaram, devido à sua performance ser extremamente baixa comparado com as linguagens existentes no mercado, como o PHP ou Java.

Porém, com a rápida evolução da *Web* nos últimos anos, a linguagem Javascript e seus motores de execução passaram por diversas melhorias, tornando viável sua execução com outros propósitos além da manipulação de páginas HTML.

Com essa nova fase no uso do Javascript, aplicações *server-side* passaram a ser implementadas, e em 2009 foi criado o primeiro ambiente de execução Javascript com este propósito: O Node.js.

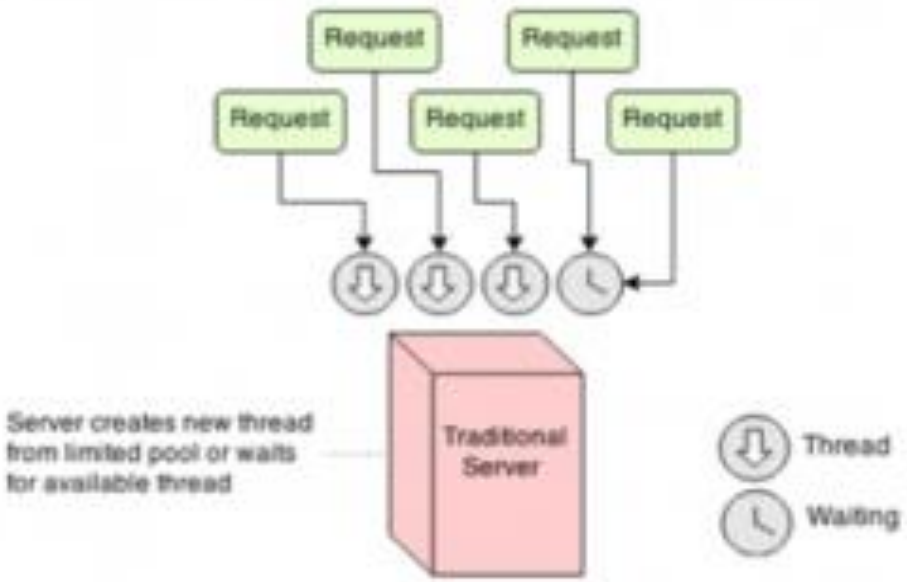
CARACTERÍSTICAS DO NODE.JS

A principal característica que diferencia o Node.JS de outras tecnologias, como PHP, Java, C#, é o fato de sua execução ser single-thread. Ou seja, apenas uma thread é responsável por executar o código Javascript da aplicação, enquanto que nas outras linguagens a execução é multi-thread.

Mas o que isso significa na prática?

Em um servidor web utilizando linguagens tradicionais, para cada requisição recebida é criada uma nova thread para tratá-la. A cada requisição, serão demandados recursos computacionais (memória RAM, por exemplo) para a criação dessa nova thread. Uma vez que esses recursos são limitados, as threads não serão criadas infinitamente, e quando esse limite for atingido, as novas requisições terão que esperar a liberação desses recursos alocados para serem tratadas.

A figura abaixo representa esse cenário em um servidor tradicional:



(Fonte: https://www.opus-software.com.br/node-js/)

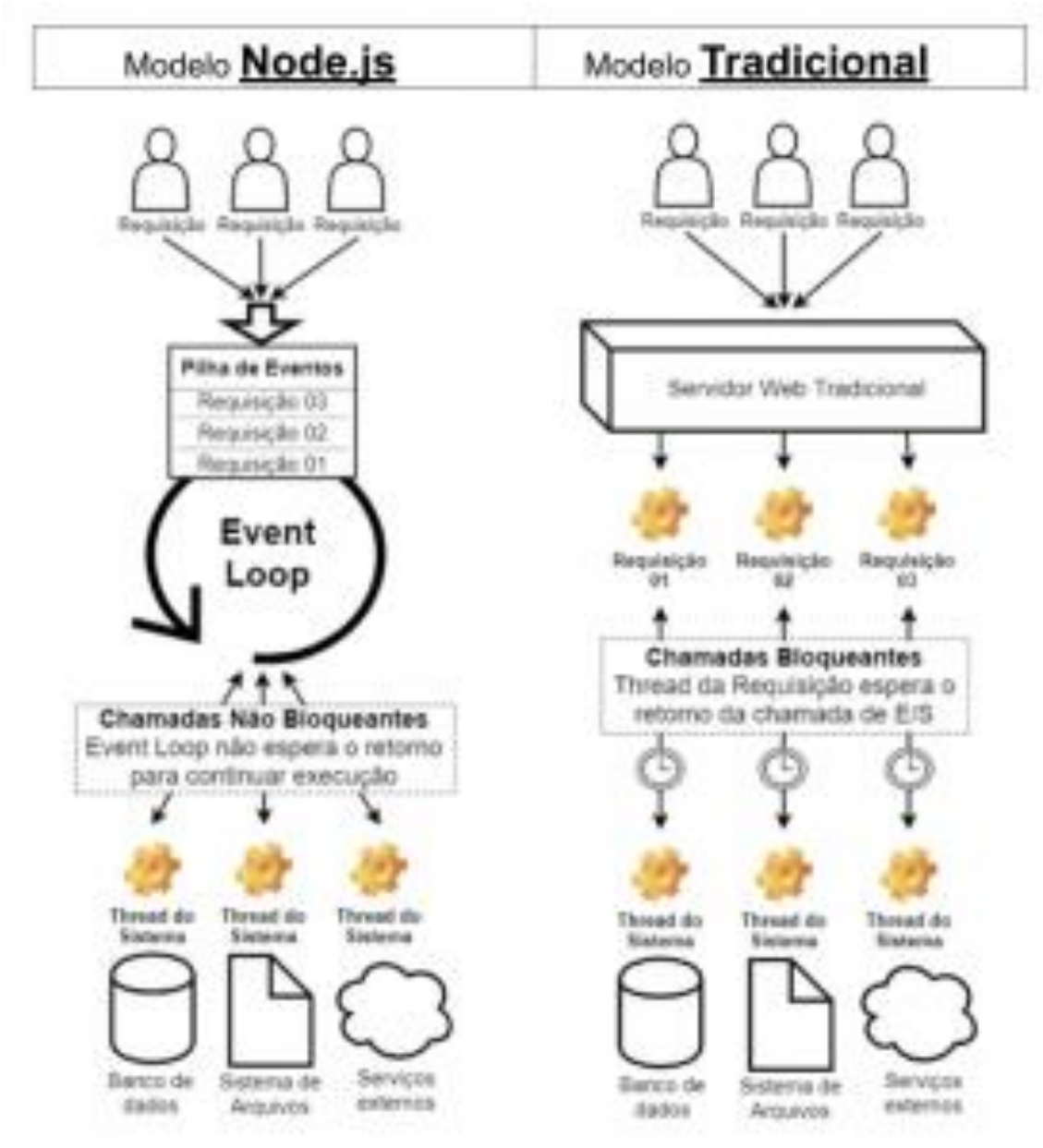
No modelo Node.js, apenas uma thread é responsável por tratar as requisições.

Essa thread é chamada de Event Loop, e leva esse nome pois cada requisição é tratada como um evento. O Event Loop fica em execução esperando novos eventos para tratar, e para cada requisição, um novo evento é criado.

Apesar de ser single-threaded, é possível tratar requisições concorrentes em um servidor Node.js. Enquanto o servidor tradicional utiliza o sistema multi-thread para tratar requisições concorrentes, o Node.js consegue o mesmo efeito através de chamadas de E/S (entrada e saída) não-bloqueantes.

Isso significa que as operações de entrada e saída (ex: acesso a banco de dados e leitura de arquivos do sistema) são assíncronas e não bloqueiam a thread. Diferentemente dos servidores tradicionais, a thread não fica esperando que essas operações sejam concluídas para continuar sua execução.

A figura abaixo representa a diferença de funcionamento de um servidor web tradicional e um Node.JS:



(Fonte: https://www.opus-software.com.br/node-js/)

No servidor Node.js, o Event Loop é a única thread que trata as requisições, enquanto que no modelo tradicional uma nova thread é criada para cada requisição. Enquanto o Event Loop delega uma operação de E/S para uma thread do sistema de forma assíncrona e continua tratando as outras requisições que aparecerem em sua pilha de eventos, as threads do modelo tradicional esperam a conclusão das operações de E/S, consumindo recursos computacionais durante todo esse período de espera.

Apesar do Node.js ser single-threaded, sua arquitetura possibilita um número maior de requisições concorrentes sejam tratadas em comparação com o modelo tradicional, que é limitado devido ao alto consumo computacional pela criação e manutenção de *threads* a cada requisição.

VANTAGENS DO USO DO NODE.JS

Flexibilidade

O NPM (Node Package Manager) é o gerenciador de pacotes do Node.js e também é o maior repositório de softwares do mundo. Isso faz do Node.js uma plataforma com potencial para ser utilizada em qualquer situação. O pacote mais conhecido se chama Express.js e é um framework completo para desenvolvimento de aplicações Web.

Leveza

Criar um ambiente Node.js e subir uma aplicação é uma tarefa que não exige muitos recursos computacionais em comparação com outras tecnologias mais tradicionais. Se utilizado em conjunto com ferramentas como o Docker, o ganho na velocidade de deploy e replicação de máquinas pode ser muito significativo e em ambientes escaláveis isso significa menos custo e mais eficiência.

Tanto sua leveza quanto flexibilidade fazem do Node.JS uma tecnologia indicada para a implementação de serviços e componentes de arquiteturas como a de microsserviços e serverless. Além disso, conta com suporte das principais empresas de produtos e serviços Cloud do mercado, como a AWS, Google Cloud e Microsoft Azure que oferecem na maioria de seus produtos suporte nativo ao Node.JS.

Produtividade da equipe

Maior repositório do mundo: O NPM fornece pacotes de código reusáveis e provavelmente aquela integração que você precisa fazer com outro sistema ou banco de dados já está implementado e disponível gratuitamente para instalar via NPM.

Mesma linguagem no frontend e backend: Javascript é a linguagem padrão para desenvolvimento web client-side. Empresas de desenvolvimento Web contam como esse know-how como um ponto de partida importante para iniciar o uso do Node.js. Além disso, esse fator pode representar ganhos de reutilização de código e criação de equipes multidisciplinares, com melhor aproveitamento de recursos.

Ambiente de inovação: Possibilidade de deploys e iterações mais rápidas, e resolução de problemas On the Fly. Isso também permite a criação de soluções próprias e inovadoras, como fez o Uber criando produtos em Node.js para resolver alguns de seus problemas.

PRINCÍPIOS

Acima de qualquer estrutura que eu possa propor aqui, vale ressaltar alguns princípios e motivações que devem estar acima de quaisquer convenções em seu código:

Ajude seu cérebro a entender o projeto: Conseguimos lidar com apenas pequenas quantidades de informação de cada vez. Por isso que usamos pastas. Por isso que criamos funções. Elas nos ajudam a lidar com a complexidade do sistema nos permitindo mexer em pequenas porções dele de cada vez.

Crie apenas os diretórios que precisa: Não saia criando dezenas de diretórios que acabarão vazios ou com apenas um arquivo dentro. Comece com o básico de pastas que você precisa e vá adicionando conforme a complexidade for aumentando. Afinal, você não compra um ônibus como primeiro veículo pensando no dia em que pode querer dar carona para muita gente, certo?

Torne fácil localizar arquivos de código: Os nomes de pasta devem ser significativos, óbvios e fáceis de entender. Código que não é mais usado, bem como arquivos antigos, devem ser removidos do projeto, e não apenas comentados ou renomeados. Além disso, todos os códigos ficam dentro de um diretório app na raiz e seus subdiretórios, para facilitar buscas posteriores, especialmente via linha de comando. Ah, e o padrão do npm é usar nomes sempre em minúsculo para as pastas, sem acentos, e com os espaços substituídos por hífens (-). Então é melhor seguir esta regra, por mais que não goste dela.

Agrupe por feature, não por responsabilidade: Esta é uma sugestão que vai contra o que a maioria de nós está acostumado, de separar os arquivos por responsabilidade (controller, model, etc). Cada feature da sua aplicação node possui uma série de arquivos que, se estiverem juntos (em uma pasta por feature), fica mais fácil de dar manutenção depois. Ex: preciso adicionar um novo campo em um model, na mesma pasta eu já posso mexer no controller que usa aquele model.

Guarde seus testes perto do seu código: Essa ideia vai de encontro à anterior. Coloque o arquivo JS de testes de cada feature na mesma pasta da feature, para facilitar os testes e correção dos bugs encontrados. Também ajuda a lembrar das features que ainda não possuem testes codificados.

Para não se perder com os nomes de arquivos a sugestão é usar uma convenção de nome baseada em sufixo, por exemplo:

o arquivo JS original é foo.js
o arquivo de teste do original é foo.test.js

Diminua o acoplamento e aumente o reuso de software: A ideia com essas duas últimas dicas é tornar sua aplicação mais modular, com cada feature sendo um módulo mais independente. Aqui a ideia permanece. Uma vez que você tenha módulos independentes, sua aplicação fica menos acoplada e a manutenção mais simples. Uma dica para aumentar o reuso de software mantendo o acoplamento baixo é não colocar código "especializado" nas suas chamadas de model e controller. Ex: após salvar um cliente no banco você precisa enviar um e-mail pra ele. Não coloque o código que envia e-mail logo após o salvamento no banco. Ao invés disso, dispare uma função de um módulo de e-mail.

Priorize configuration over convention: Não faça coisas mágicas como assumir que o nome de um arquivo é x ou que os uploads vão parar sempre em uma pasta y. Isso é frágil e facilmente quebra sua aplicação quando alguma coisa muda e você não lembra de todas as convenções que criou para sua arquitetura. Programe de maneira que o código em si possa fazer com que o programador entenda todo o projeto.

Convencione nomes de arquivos no formato 'lower-kebab-case': Use apenas minúsculas e '-'como separador entre palavras. Isso evita problemas de sistemas de arquivos em SOs diferentes que sua aplicação Node possa rodar. Ex: cadastro-cliente.js

ESTRUTURA E PERFORMANCE

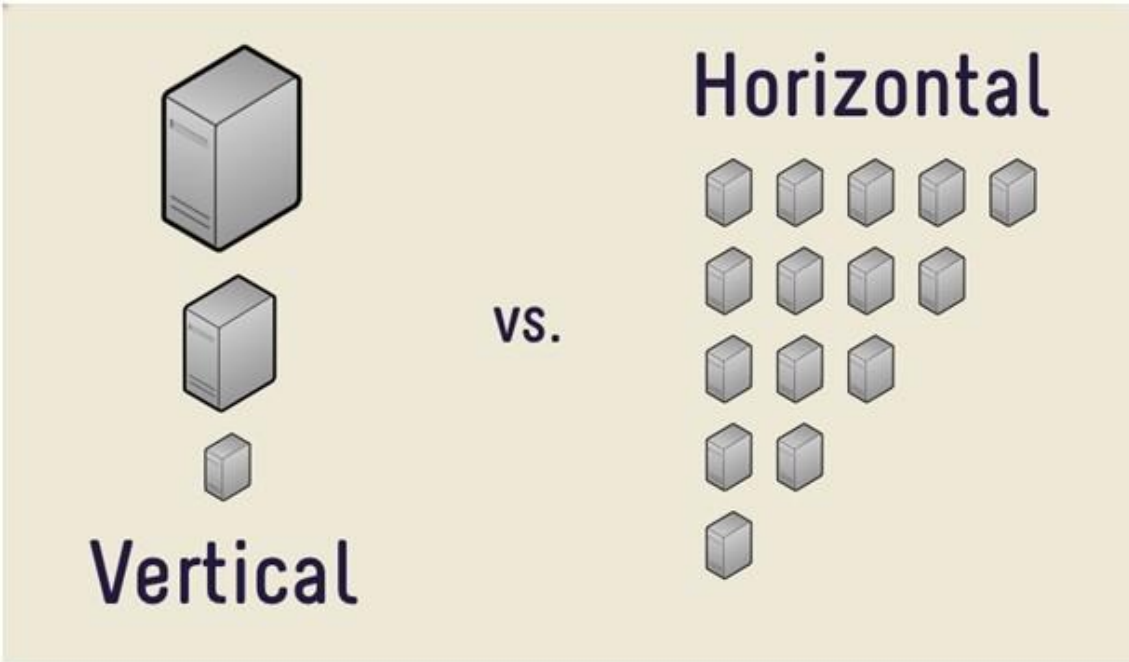
Mais popular a cada dia no mundo do software, sua arquitetura *single thread* (Thread Única) com um sistema não bloqueante permite atingir um número absurdo de requisições por segundo com pouco recurso computacional.

De forma simples, isso se dá porque Node.js usa um loop de eventos de i/o (input/output) para processar requisições de forma intermitente, assim que um evento é finalizado o mesmo dispara um callback (função de retorno), assim o evento é posto novamente na fila e praticamente elimina-se a ociosidade no processamento.

Em plataformas mais convencionais usa-se *threads*, porém enquanto elas aguardam uma resposta do evento poderiam estar fazendo outras tarefas, mas ela fica lá “matando tempo” e bloqueando outros eventos que estão na fila, além de exigir um controle por parte do Sistema Operacional para que todas as threads sejam gerenciadas. Isso requer mais recurso computacional e resulta em perda de performance.

ESCALABILIDADE

Node.js foi pensado para ser altamente escalável horizontalmente, por exemplo: Ao invés de investir em discos SSD, Memória ROM, RAM, você pode simplesmente virtualizar a sua máquina e/ou adicionar mais máquinas no mesmo ponto físico ou não, tornando assim sua escalabilidade muito mais simples e barata e eficaz.



(Fonte: https://www.linkedin.com/pulse/uma-arquitetura-para-web-com-nodejs-graphql-postgres-matheus)

Repare na imagem acima, na escala vertical os recursos aumentam de tamanho, ou seja, aumenta-se espaço em disco, memória, etc. Na escala horizontal você multiplica os recursos.

ESTENSIBILIDADE E PACOTES

Se você for se aventurar e programar de forma nativa no Node.js, vai ver que as coisas não são assim tão simples como você provavelmente viu em algum código na internet, porém, quando instalamos o Node.js temos a disposição o “npm” (Node Package Manager / Gerenciador de Pacotes do Node), com ele você tem acesso a milhares de packages que podem ser adicionados a sua aplicação. Todos os pacotes estão disponíveis em que é o repositório oficial de pacotes Node. Isso abstrai muito da complexidade do desenvolvimento em Node.

É preciso ter um pouco de cuidado ao utilizar esses pacotes, é importante conferir quantas pessoas já o utilizam, número de comentários e as informações disponíveis sobre o pacote no Github como commits, pull requests, contribuintes, etc. Isso porque qualquer um pode desenvolver e publicar um pacote Node.

ARQUITETURA LIMPA

Arquitetura Limpa, do termo inglês *Clean Architecture*, é um padrão arquitetural usado na engenharia de software. Esse padrão foi conceituado por Robert Martin, mais conhecido como Uncle Bob, um engenheiro de software famoso e um dos autores do Manifesto Ágil (link), além de diversos livros na área. O conceito foi apresentado no artigo em seu blog em 2012 (link) e foi baseado em uma arquitetura já conhecida, a Arquitetura Hexagonal (link), conceituada em 2005. Mais tarde, em 2017, ele aprofundou-se no tema em seu livro: Arquitetura Limpa: O guia do artesão para estrutura e design de software (link).

A ideia dessa arquitetura é criar componentes fracamente acoplados que possam ser conectados através de portas e adaptadores, o que permite o isolamento das regras de negócio da aplicação (ex: entidades, casos de uso) de preocupações externas (ex: frameworks, banco de dados). A lógica de negócios não deve depender de GraphQL ou REST por exemplo, ou se os dados estão sendo obtidos por um banco de dados ou um arquivo CSV simples. Essa separação tem, dentre outros benefícios, auxiliar na mudança de componentes e facilitar testes automatizados.

A Arquitetura Limpa casa muito bem com o DDD (Domain-Driven Design) (link). Irei utilizar a notação usada pelo DDD para definir as camadas da aplicação, que são: Domínio, Aplicação, Interfaces e Infraestrutura. Cada uma dessas camadas é definidas da seguinte maneira:

Domínio: A camada de domínio é o coração do software, reunindo as regras cruciais de negócios da empresa inteira. É o local onde as entidades e objetos de valor viverão. De acordo com Uncle Bob: “Uma entidade pode ser um objeto com métodos ou um conjunto de estruturas de dados e funções. Isso não importa, contanto que as entidades possam ser usadas por muitas aplicações diferentes na empresa.”.

Aplicação: Essa camada contém as regras de negócio específicas da aplicação. Na Arquitetura Limpa, Uncle Bob descreve os casos de uso como

sendo as principais funcionalidades da aplicação. Esses casos de uso utilizam as entidades a fim de atingir seus objetivos, criando assim uma dependência nelas.

Interfaces: Aqui estão os adaptadores. Adaptadores são criados para isolar as entidades e casos de uso das ferramentas utilizadas na aplicação (frameworks e drivers), convertendo os dados para o formato que mais convém às camadas mais externas. Esses adaptadores, de acordo com Uncle Bob, devem funcionar pelo conceito de Interfaces de programação orientada a objetos nos casos de uso, como apresentado no diagrama do canto inferior da imagem acima. Como em Javascript não existem interfaces, isso pode ser feito através de injeção de dependência, que será explicado melhor nas seções seguintes.

Infraestrutura: A infraestrutura consiste em tudo o que existe independentemente da aplicação: bibliotecas externas, mecanismo de banco de dados, servidor de aplicação, filas de mensagens e assim por diante.

EXPRESS

Express é um framework para desenvolvimento de aplicações Web, minimalista e flexível, que roda sobre a plataforma Node.js (Express - web application framework for node, 2014).

Provendo um conjunto robusto de funcionalidades para a construção de aplicações Web, sejam elas Spa (Single-page Applications), aplicações com múltiplas páginas, ou aplicações híbridas. Com diversos métodos utilitários HTTP e de middleware disponíveis, o Express torna fácil a criação de Web APIs amigáveis de forma rápida e robusta.

O Express provê uma pequena camada de funcionalidades fundamentais para aplicações Web, porém, sem obscurecer as funcionalidades já disponíveis no Node.js.

Quando trabalhamos com uma plataforma nova como o Node.js, é muito comum ficarmos em dúvida de como proceder com a escala de nossas aplicações e principalmente com a arquitetura da mesma, conforme vão crescendo.

Além disso, considerando os fundamentos do Node.js, o fato de trabalhar com Javascript e sua inerente dinamicidade e pouco apreço por tipagem, orientação à objetos e muito mais, torna a transição de programadores mais tradicionais (Java, C#, etc) para esta plataforma um tanto confusa.

Aplicações web não são sempre a mesma coisa e, da mesma forma, não há uma única arquitetura e/ou estrutura de código que possa ser aplicada à todas aplicações express.js.

Sim, é possível criar grandes aplicações em Node.js uma vez que grandes empresas estão fazendo exatamente isso nos últimos anos, como Walmart, Netflix e Uber, só para citar algumas. A questão aqui é: sua aplicação realmente é grande e precisa de uma arquitetura como a deles?

Se sua aplicação é pequena, você não precisa de uma estrutura complexa de diretórios. Mantenha sua aplicação simples, garanta que os arquivos mais importantes estarão na raiz do diretório e está feito.

Agora, se sua aplicação é gigantesca, em algum ponto você pode precisar quebrá-la em pacotes NPM separados (semelhante ao que fazemos com as class libraries em Java e C#).

Em geral, o que se recomenda a fazer com Node.js é sempre trabalhar com pacotes pequenos, especialmente para bibliotecas, uma vez que o fato de usar a linguagem Javascript não ajuda muito em manter o controle e previsibilidade de funcionamento em uma aplicação conforme ela fica muito grande. Sendo assim, conforme sua aplicação for crescendo e partes do seu código comecem a se tornar reutilizáveis, mova-os para um pacote como se fosse um subsistema ou biblioteca, tornando-o independente no NPM.

CASOS DE USO MAIS COMUNS

Aplicações em Tempo Real

Um exemplo comum é uma aplicação de conversa (chat). Tal aplicação exige muito pouco processamento e basicamente consiste em transferir as mensagens de um lado para outro.

Ambientes Escaláveis

O Node.js é bastante indicado para ambientes escaláveis (com grande número de conexões concorrentes), já que tem potencial para suportar um número maior de conexões simultâneas do que servidores tradicionais.

Camada de Entrada do Servidor

O Node.js faz pouco processamento de dados e apenas passa a requisição para frente, se comunicando com serviços de backend.

Mocks e Protótipos

Por utilizar uma linguagem bastante conhecida no mundo Web, o Node.js possibilita criar mocks e protótipos de APIs e serviços de *backend* com grande rapidez, podendo assim simular a comunicação com um serviço externo, por exemplo.

API com NoSQL por trás

As base de dados NoSQL são baseadas em JSON (JavaScript Object Notation), portanto, sua comunicação com Node.js é bastante intuitiva. Com isso, não é necessário converter modelos de dados, por exemplo, pois os mesmos objetos JavaScript armazenados na base de dados podem ser enviados para o front-end sem a necessidade de nenhum tipo de tratamento ou conversão.

CONSIDERAÇÕES FINAIS

Node.js está em uma incrível ascensão, é uma ferramenta extremamente poderosa e que está mudando muita coisa no mundo do software.

Suas grandes vantagens são extensibilidade, escalabilidade e performance, porém é necessária uma boa pré-avaliação antes de tomar a decisão de fazer algo em Node, pois não é a solução ideal para todos os casos.

Com o passar dos anos, a tecnologia Node.js já rivaliza com tecnologias tradicionais com como o ASP.NET, JSF, Ruby, PHP e Python em ambientes dinâmicos como *startups* e empresas de produtos Web. Mesmo empresas tradicionais no Brasil já têm começado a experimentar essa tecnologia bastante promissora.

Ao mesmo tempo, algumas pessoas ainda perguntam: “É possível fazer em Node.JS uma aplicação com acesso a banco de dados com controle transacional, suporte a APIs REST, micro-serviços, com segurança forte, tolerância a falhas e operação em cluster, entre outras preocupações técnicas corporativas?”.

A resposta, retumbante, é sim.

Para isso basta você acrescentar novos módulos ao node com o comando npm install -g <NomeModulo>. Isso é porque o Node.js vem pelado de fábrica e você deve incluir módulos conforme necessário. Essa abordagem minimalista, que permite criar ambientes Web muito leves, é um dos fatores de sucesso do Node.JS

REFERÊNCIAS

CARVALHO. Afonso A P. **Arquitetura e desenvolvimento de uma aplicação web distribuída baseada em javascript.** Rio de Janeiro, RJ – Brasil. 2014.

DUARTE, Leo. **Boas práticas de arquitetura com NodeJS + Express**. 2017.

LENON. **Node.js – O que é, como funciona e quais as vantagens**. 2018.

MENDES, Marco. **Arquitetura de Software: Não Alimente Seus Paquidermes**. 2017.

SCHUMACHER, Matheus Schafer. **Node.js - Uma descrição do ponto de vista da Arquitetura de Software**. 2019.

YODER, Joseph, apud FOOTE, Brian. **Uma Breve Introdução à Arquitetura Limpa com Node.js**. 2021.